

Guia de Estudo — Bin Packing: Garantias de Pior Caso × Desempenho Médio

Disciplina: Análise e Projeto de Algoritmos (CESUPA, 2026) **Aluno:** Benjamin Yuji Suzuki **Artigo:** *Garantias de Pior Caso versus Desempenho Médio de Heurísticas Clássicas para o Bin Packing Unidimensional: um Estudo Experimental* **Repositório:** <https://github.com/Benjamin-Yuji-Suzuki/bin-packing-heuristics> **PDF compilado:** [artigo/artigo-parcial.pdf](#) (9 páginas, formato SBC)

0. O resumo do artigo, decodificado (leia antes de tudo)

O resumo acadêmico comprime 15 páginas em ~10 frases densas. Aqui vai cada frase traduzida, com os básicos, usando o exemplo-fio `[0.7, 0.6, 0.4, 0.2]` (OPT = 2 bins).

1. **"O problema pede o número mínimo de recipientes de capacidade unitária"** — tenho itens de tamanhos e caixas de capacidade 1; quantas caixas no mínimo? No exemplo: `{0.7, 0.2} + {0.6, 0.4}` = 2 caixas, e 1 é impossível (soma $1.9 > 1$). **OPT = 2.**
2. **"NP-difícil"** — não existe algoritmo rápido conhecido que ache o mínimo exato (crê-se que não exista; provar é o P vs NP). Com 16.000 itens, ninguém calcula o ótimo. Por isso a gente desiste do ótimo e usa plano B.
3. **"Heurísticas clássicas... escolha padrão na prática"** — regras de dedo simples ("coloca na primeira caixa que couber") que dão resposta boa instantânea. Sistemas reais (memória, corte, agendamento) ainda usam elas 50 anos depois.
4. **"Garantias de pior caso: razão 2 (NF), 1.7 (FF/BF), 11/9 OPT + 6/9 (FFD)"** — cada heurística vem com promessa matemática: "nunca uso mais que $X \times$ o ótimo". FFD: no máximo 22% a mais de caixas. O "+6/9" é um restinho fixo em cima, cujo valor exato ficou em aberto 34 anos (1973 → 2007, Johnson → Dósa).
5. **"A teoria diz pouco sobre o comportamento típico"** — a garantia responde "quão ruim PODE ser" (colisão a 60 km/h), não "quão ruim É no dia a dia". As garantias só são atingidas por instâncias construídas de propósito pra serem cruéis. Essa é a lacuna que o artigo ataca: medir o caso comum.
6. **"Experimento controlado: 4 distribuições × 5 tamanhos × 10 repetições"** — distribuição = de onde os tamanhos são sorteados (ex.: só itens médios 0.26–0.50 = caso difícil); tamanhos dobrando (1k → 16k) = ver a escala; 10 reps = média contra instância azarada. $4 \times 5 \times 10 \times 5$ algoritmos = 1.000 execuções.
7. **"Razão contra o limite inferior L2 de Martello–Toth"** — o ótimo é incalculável, então compara-se com um **piso provado** ($L2 \leq OPT$, como saber que alguém tem pelo menos 1,70m sem medir). Se o algoritmo usou 100 caixas e $L2 = 98$, a razão é 1.02. Detalhe de defesa: como $L2 \leq OPT$, a razão $A/L2$ é sempre $\geq$ à razão real A/OPT — os números do artigo são pessimistas; a qualidade real é ainda melhor.

8. **"Razões empíricas muito abaixo das garantias (FFD 1.001 vs 1.222)"** — na prática o FFD erra 0.1%, não 22%. Resposta direta à lacuna da frase 5. (E isso valida teoremas clássicos: Coffman 1980, Bentley 1984 — ver seção 7.1.)
9. **"3-partição é a mais difícil"** — só itens médios = sobras que nenhum item preenche (sem "argamassa" pequena). Razões ~1.10–1.13 em vez de ~1.00–1.02.
10. **"Implementações $O(n^2)$ proibitivas; NF linear 170× mais rápido, porém 30% pior"** — FF/BF varrem todas as caixas por item ($n \times n$); NF olha só a atual (n). Qualidade tem preço em tempo: o artigo mede esse trade-off.
11. **"Contrexemplos em que ordenar piora o First Fit"** — FFD = FF + ordenar. A intuição diz que ordenar ajuda; achamos instâncias onde PIORA (os grandes se espalham e criam sobras inúteis; a ordem original misturava melhor por sorte).

O resumo em 1 parágrafo de gente: bin packing = colocar itens em caixas de capacidade 1 com o mínimo de caixas; achar o mínimo é inviável (NP-difícil), então usa-se 5 regras simples com promessas do tipo "nunca pioro mais que 22%"; mas essas promessas são pro pior caso imaginável, e ninguém sabe o que acontece normalmente; então implementamos tudo em Rust, rodamos 1.000 testes controlados medindo qualidade (contra um piso teórico) e tempo; descobrimos que na prática as regras erram quase nada (0.1%, não 22%), que o pior cenário é só itens-médios, que as regras espertas ficam lentas demais em listas grandes (a simples é 170× mais rápida com só 30% mais caixas), e que ordenar antes às vezes PIORA o resultado.

1. Como ler o artigo (seção por seção)

Seção	O que faz	O que o professor vai perguntar
§1 Introdução	Define o problema, motiva, declara objetivo	"Por que bin packing é difícil?" (NP-difícil)
§2 Fundamentação	Define notação, prova NP-completo, apresenta 5 heurísticas + garantias	"A garantia do FFD é tight?" (sim, Dósa 2007)
§3 Trabalhos Relacionados	Separa literatura em 3 frentes (pior caso, random-order, experimento)	"Qual a diferença entre Kenyon e Balogh?" (Kenyon = BF random-order, Balogh = online geral)
§4 Metodologia	Distribuições, L2, protocolo; inclui a Subseção 4.5 Resultados Preliminares	"Sua razão é conservadora?" (sim, $A/L2 \geq A/OPT$)
§5 Considerações Parciais	Fechamento do 1º bimestre; resume descobertas sobre NF e 3-partição	"O que concluiu até agora?" (NF é rápido e bom; garantias são pessimistas)
§6 Cronograma	Lista o que falta ($n=64k$, B&B, random-order, versão final)	"O que você faria no 2º bim?" (responder §6)

Seção	O que faz	O que o professor vai perguntar
§7 Declaração de IA	Tabela detalhando ferramentas usadas e revisão humana	"O que a IA fez vs você?" (leitura da tabela)

2. O problema: definição formal

Entrada: lista $L = (s_1, s_2, \dots, s_n)$ com $s_i \in (0, 1]$. **Saída:** partição de L em grupos ("bins") tal que a soma de cada grupo ≤ 1 e o **número de grupos é mínimo**.

- **Solução viável:** qualquer partição que respeite a capacidade.
- **OPT(L):** custo da solução ótima (número mínimo de bins).
- **Aplicações reais:** alocar arquivos em blocos de disco, tarefas em lotes de processamento, cortes de matéria-prima, carregamento de cargas.

Razão de aproximação: para um algoritmo A , a razão é $A(L)/OPT(L)$. Dizemos que A é **p-aproximativo** se $A(L) \leq p \cdot OPT(L) + c$ para toda L (c constante). Quanto mais perto de 1, melhor.

Por que não calcular OPT direto? Porque é NP-difícil — instâncias grandes são inviáveis. Por isso medimos contra um **lower bound** (L_2 , Seção 5): como $L_2 \leq OPT$, a razão A/L_2 é sempre $\geq$ razão real A/OPT . Ou seja: nossas razões empíricas são estimativas **conservadoras** (pessimistas) — a qualidade real é ainda melhor do que reportamos. *(Essa é uma pegadinha clássica de defesa — decora!)*

3. Por que é difícil: NP-completude (Teorema 1 do artigo)

Teorema: Bin Packing (versão de decisão: "dá com $\leq k$ bins?") é NP-completo.

Prova (esboço, via redução da 3-PARTIÇÃO):

1. **Pertence a NP:** dado um certificado (a partição), verifica-se em tempo linear se toda soma ≤ 1 e se usa $\leq k$ bins.
2. **NP-dureza:** reduzimos a 3-PARTIÇÃO, que é NP-completa **no sentido forte** (Garey & Johnson 1979): dados $3m$ inteiros $a_1 \dots a_{3m}$ com $\sum a_i = mB$ e $B/4 < a_i < B/2$, decidir se existe partição em m tripletas de soma exatamente B .

Construímos a instância de bin packing: itens $s_i = a_i/B$ (capacidade normalizada pra 1), e perguntamos: existe solução com $\leq m$ bins?

- Como $s_i > 1/4 \rightarrow$ cabem **no máximo 3 itens** por bin.
- Como $s_i < 1/2 \rightarrow$ **dois itens somam < 1** , então nenhum bin fica "meio cheio" com 2.
- A soma total é $\sum s_i = m \rightarrow$ qualquer solução com m bins tem todos os bins com soma **exatamente 1**, cada um com **exatamente 3 itens** — ou seja, uma 3-partição.

Responder bin packing responde 3-PARTIÇÃO. ■

Por que "no sentido forte" importa: problemas NP-difíceis no sentido forte não admitem FPTAS (esquema de aproximação quase-exato em tempo polinomial) a menos que $P=NP$. É por isso que a aproximação prática se dá por heurísticas com garantia (Williamson & Shmoys 2011).

4. As cinco heurísticas (com exemplo numérico)

Exemplo de trabalho: $L = [0.7, 0.6, 0.4, 0.2]$ (ordem de chegada). $OPT = 2$.

4.1 Next Fit (NF) — $O(n)$

Mantém **um único bin aberto**. Se o item não cabe, **fecha** o bin (para sempre) e abre outro. Nunca reabre bin fechado.

```
0.7 → bin1 (sobra 0.3)
0.6 → não cabe em 0.3 → fecha bin1, bin2 (sobra 0.4)
0.4 → cabe em 0.4 → bin2 (sobra 0.0)
0.2 → não cabe em 0.0 → fecha bin2, bin3
Resultado: 3 bins
```

Garantia: $NF(I) \leq 2 \cdot OPT(I)$ (Johnson et al. 1974). Intuição da prova: qualquer bin fechado pelo NF tem sobra menor que o item que o fechou, então **dois bins consecutivos sempre somam mais que 1** → $NF \leq 2 \cdot [soma\ total] + 1 \leq 2 \cdot OPT$.

Instância ilustrativa do desperdício (do nosso teste): (0.6, 0.5) repetido n vezes → $NF = 2n$ bins; $FF = 1.5n$. NF nunca consegue fechar um par ($0.6+0.5 > 1$) e nunca reabre.

4.2 First Fit (FF) — $O(n^2)$ direto

Coloca o item no **primeiro** bin (em ordem de abertura) onde couber; abre novo se necessário. Reabre/consulta todos os bins abertos.

```
0.7 → bin1 (sobra 0.3)
0.6 → bin1 não cabe → bin2 (sobra 0.4)
0.4 → bin1 não cabe (0.3), bin2 cabe (0.4) → bin2 (sobra 0.0)
0.2 → bin1 cabe (0.3) → bin1
Resultado: 2 bins = OPT ✓
```

Garantia: $FF(I) \leq 1.7 \cdot OPT(I)$ (Johnson et al. 1974), tight.

4.3 Best Fit (BF) — $O(n^2)$ direto

Coloca o item no bin com **menor espaço restante** onde ainda couber (aperta o item no bin mais cheio possível — minimiza fragmentação).

```
0.7 → bin1 (sobra 0.3)
0.6 → só bin2 é viável → bin2 (sobra 0.4)
0.4 → menor sobra viável é 0.4 (bin2) → bin2 (sobra 0.0)
0.2 → sobra viável: 0.3 (bin1) → bin1
Resultado: 2 bins = OPT ✓
```

Garantia: $\text{BF(I)} \leq 1.7 \cdot \text{OPT(I)}$ (Johnson et al. 1974), tight.

4.4 First Fit Decreasing (FFD) — $O(n \log n) + O(n^2)$

Ordena os itens do maior pro menor e aplica FF. A intuição: itens grandes primeiro definem a "espinha dorsal"; itens pequenos no final preenchem os buracos (argamassa).

```
Ordenado: [0.7, 0.6, 0.4, 0.2]
0.7 → bin1 (sobra 0.3)
0.6 → bin2 (sobra 0.4)
0.4 → bin2 (sobra 0.0)
0.2 → bin1 (sobra 0.1)
Resultado: 2 bins = OPT ✓
```

Garantia: $\text{FFD(I)} \leq 11/9 \cdot \text{OPT(I)} + 6/9$ (Dósa 2007 — fecha um problema aberto por 34 anos!).
 $11/9 \approx 1.222$.

A história do 11/9 (conta boa pra defesa):

- 1973: Johnson prova $11/9 \cdot \text{OPT} + 4$ na tese de doutorado
- ~1980: refinamentos posteriores melhoram a constante
- **2007: Dósa fecha em 6/9 e prova que é tight** (não dá pra melhorar)

34 anos pra fechar UMA constante aditiva — mostra o quanto essa análise é delicada.

4.5 Best Fit Decreasing (BFD) — $O(n \log n) + O(n^2)$

Ordena decrescente + BF. **Mesma garantia do FFD:** $11/9 \cdot \text{OPT} + 6/9$.

 **O achado do nosso artigo: ordenar pode PIORAR!**

$\text{FFD(I)} \leq \text{FF(I)}$ NÃO é teorema. Existe contraexemplo empírico no nosso repositório (distribuição 3-partição, $n=50$, semente 1003) onde FFD usa **mais bins que FF**. Mecanismo: ordenar coloca todos os itens grandes primeiro, que se espalham criando sobras pequenas demais para os

médios; sem ordenar, a mistura casual combina melhor. Descobrimos isso com um **teste de propriedade** (**cargo test**) que tentava validar " $FFD \leq FF$ " e **falhou** — a IA tinha assumido a premissa errada, o teste desmentiu.

5. O lower bound L2 de Martello–Toth (1990)

Para medir qualidade sem conhecer OPT, usamos **L2**, um limite inferior ($L2 \leq OPT$).

L1 (trivial): [soma dos tamanhos] — todo bin tem capacidade 1, então precisa de pelo menos isso. Fraco porque ignora fragmentação.

L2 (refinado): para cada corte $\alpha \in (0, 0.5]$:

- Itens **grandes** ($s > 1-\alpha$): cada um exige um bin próprio $\rightarrow$ contam 1 cada.
- Itens **médios** ($\alpha \leq s \leq 1-\alpha$): não podem dividir bin com os grandes $\rightarrow$ precisam de [soma dos médios] bins entre si.
- $L2(\alpha) = |\text{grandes}| + [\text{soma dos médios}]$.

O L2 final é o **máximo** sobre os α candidatos (nossa implementação: $\alpha = 0.5$ fixo + os tamanhos dos itens ≤ 0.5 , com dedup — o máximo ocorre nesses pontos).

Exemplo: $[0.6, 0.6, 0.6] \rightarrow L1 = \lceil 1.8 \rceil = 2$, mas L2 com $\alpha=0.5$: os três são grandes (>0.5) $\rightarrow$ 3 bins. OPT = 3 (nenhum par cabe junto). L2 pega o que L1 deixa passar.

6. O experimento: desenho e números

Protocolo: 4 distribuições $\times$ 5 tamanhos ($n = 1.000, 2.000, 4.000, 8.000, 16.000$) $\times$ 10 repetições $\times$ 5 algoritmos = **1.000 execuções**. Sementes determinísticas (splitmix64 misturado com n) $\rightarrow$ 100% reprodutível.

As 4 distribuições (por que cada uma existe):

Distribuição	Definição	Intuito
uniforme_continua	$U[0,1]$	caso base: mistura de tudo
uniforme_discreta_100	$U\{1/100, \dots, 1\}$	discretizada em centésimos
tres_particao	$U\{0.26, \dots, 0.50\}$	deliberadamente difícil: todos os itens em $(1/4, 1/2]$ — sem itens pequenos para servir de "argamassa"; no máximo 3 por bin
falkenauer_u120	$U\{1/10, \dots, 1/2\}$ (média 0.275)	o benchmark clássico da área (Falkenauer 1996)

Métricas: qualidade = $A(I)/L2(I)$; tempo = relógio de parede em μs , só o empacotamento, 10 reps com média.

Ambiente: Intel Core i5-13420H (12 threads), 16 GB RAM, Linux Mint 22.3, Rust 1.98 **release** (LTO). Single-thread por medição.

Resultados principais (razão média A/L2, tempo em μ s no $n=16.000$ uniforme discreta):

Algoritmo	U[0,1]	U{1/100}	3-partição	Falkenauer	tempo (μ s)	Garantia teórica
NF	1.328	1.323	1.235	1.198	146	2.0
FF	1.021	1.017	1.131	1.032	25.527	1.7
BF	1.011	1.009	1.131	1.031	48.228	1.7
FFD	1.001	1.000	1.105	1.014	30.774	1.222
BFD	1.001	1.000	1.105	1.014	71.068	1.222

Ajuste assintótico: regressão log-log ($\log t = \beta_0 + \beta_1 \log n$) confirma expoente ≈ 1 para NF e ≈ 2 para FF/BF — a curva empírica casa com a teoria.

7. Os achados (o coração do artigo — DECORE)

1. **As garantias são drasticamente pessimistas.** FFD médio 1.001–1.014 nas distribuições comuns, contra a garantia 1.222. Mesmo na pior distribuição medida, nenhuma heurística passou de 1.24. A distância entre "quão ruim PODE ser" e "quão ruim TÍPICAMENTE é" é enorme.
2. **3-partição é consistentemente a distribuição mais difícil** para todas as heurísticas (FF: 1.131 lá vs 1.017 no uniforme). Confirma que a **ausência de itens pequenos** é o fator dominante de dificuldade — sem argamassa, os buracos não fecham.
3. **O tempo separa mais que a qualidade.** Em $n=16.000$: NF é **170× mais rápido** que FF e **487×** mais rápido que BFD, com qualidade só ~30% pior. Em cenário real com milhões de itens, NF (ou FF com estrutura melhor) pode ser a escolha racional. Trade-off qualidade × custo é a decisão de engenharia.
4. **Ordenar pode piorar** (contraexemplos $FFD > FF$, Seção 4.4) — a ordenação prévia não é monotonicamente benéfica, achado que a literatura de garantias não enfatiza.

Validação externa: a conclusão "MUITO pessimistas" bate com a literatura

A resposta curta do projeto **não é achado isolado** — é a confirmação experimental de teoremas clássicos da análise probabilística. Nossas medições batem com a teoria:

Nossa medição	O que a teoria prevê	Fonte
NF 1.328 (uniforme contínua)	razão esperada exata $4/3 = 1.3333$	Coffman, So, Hofri & Yao 1980
FF 1.017–1.021 (uniforme)	FF é assintoticamente ótimo na expectativa (razão $\rightarrow 1$)	Bentley et al. 1984 (STOC)
FFD 1.001 com excesso constante	excesso esperado $O(1)$ p/ itens $\leq 1/2$ (desperdício não cresce com n)	Bentley et al. 1984
3-partição a mais difícil (1.105)	só distribuições simétricas em torno de $1/2$ têm desperdício sublinear; as demais, linear	Bentley et al. 1984 / Lueker 1982

Bônus histórico (argumento de justificativa): o paper **experimental** (Bentley et al. 1983, Allerton) veio **antes** e inspirou o **teórico** (STOC 1984) — os experimentos sugeriram "essas heurísticas são assintoticamente ótimas, contradizendo conjecturas da época", e isso virou teorema no ano seguinte. Ou seja: o tipo de trabalho que este artigo faz (experimento controlado de comportamento médio) é historicamente o que gerou descobertas nessa área.

Nota de honestidade — o que é nosso e o que é da literatura (NÃO inflar na defesa):

Classificação	Definição	Nosso caso
Descoberta	resultado novo, desconhecido da literatura	nenhum
Redescoberta independente	chegar a resultado conhecido sem conhecer a fonte	nenhum — a 3-partição foi incluída NO DESENHO porque a literatura a aponta como difícil
Validação/replicação	medir o que a teoria prevê e confirmar	NF 1.328 vs $4/3$ (Coffman 1980)
Confirmação de desenho	observar o que o experimento foi construído para observar	3-partição e Falkenauer difíceis

- A frase certa é sempre "meu experimento **valida/replica** comportamento previsto por teoremas clássicos" — isso valida a **implementação** e dá credibilidade ao desenho. Nunca "eu descobri/redescobri".
- O que é genuinamente nosso: o desenho conjunto qualidade $\times$ tempo (o gap identificado na literatura), a validação cruzada entre linguagens (0 divergências em 9.000) e a documentação dos contraexemplos FFD > FF — valor de **documentação**, não de descoberta.

Fala pronta: "Minha conclusão de que as garantias são muito pessimistas não é isolada — é confirmação experimental de teoria clássica: Coffman et al. provaram razão esperada $4/3$ para o NF e eu medi 1.328; Bentley et al. provaram que o FF é assintoticamente ótimo na expectativa e o

FFD tem excesso esperado constante, e eu medi 1.017 e 1.001. Meu diferencial é medir qualidade e tempo juntos, e documentar os contraexemplos $FFD > FF$.

8. Benchmark entre linguagens (o bônus demonstrativo)

Metodologia (fair play):

- Rust **exporta** as 200 instâncias em binário (f64 little-endian); C, C++ e Python **consomem exatamente os mesmos bytes** — mesmos itens, mesmos algoritmos, mesmo protocolo (aquecimento descartado + 1 medição por instância).
- Otimização máxima em todos: Rust `--release` + `target-cpu=native`; C/C++ `gcc/g++ 13.3 -O3 -march=native`; CPython 3.11.
- Variantes "bins-only": as 4 linguagens fazem exatamente o mesmo trabalho (só contam bins, sem rastrear item → bin).
- **Validação cruzada: bins idênticos em 100% das 9.000 comparações.**
- Números: mediana de 3 execuções completas, **máquina ociosa** (a 1ª rodada foi descartada — estava contaminada por atividade simultânea).

Resultado (n=16.000, uniforme discreta, mediana de 3 execuções):

Algoritmo	Rust (µs)	C (µs)	C++ (µs)	Python (µs)	rust/C
NF	38	67	64	559	0.57×
FF	26.847	25.381	24.965	1.412.408	1.06×
BF	31.202	33.714	35.703	1.761.366	0.93×
FFD	28.591	27.961	27.457	1.527.636	1.02×
BFD	40.784	41.670	41.328	2.551.558	0.98×

Leitura: Rust ganha de C/C++ em NF (1.7× mais rápido) e BF, empata em BFD, fica 2–8% atrás em FF/FFD (perto do ruído). Python: ~15× atrás no linear, ~53–63× nos quadráticos.

Como o Rust chegou lá: a 1ª versão Rust media 1.3–3× o tempo do C. Dois problemas: (a) fazia **trabalho a mais** (rastreava item → bin, o que as outras não faziam); (b) usava indexação com bounds check. Correções: variantes bins-only (mesmo trabalho)

- idiomas vetorizáveis — iteradores (sem bounds check), `Vec::with_capacity`, `f64::total_cmp` no sort (comparação por bits, sem branch de NaN), e Best Fit branchless em duas passadas (redução de mínimo mascarado, vetoriza como `minpd`). Zero `unsafe`.

9. As referências: o básico e o intuito de cada uma

PRIORIDADE 1 — leitura obrigatória (o professor espera domínio)

[1] **Johnson et al. (1974)** — *Worst-Case Performance Bounds for Simple One-Dimensional Packing Algorithms*. SIAM J. Computing 3(4):299–325.

- **O que é:** o artigo **seminal**. Estabelece garantias de NF ($2 \cdot \text{OPT}$), FF e BF ($1.7 \cdot \text{OPT}$) e prova que são **tight**.
- **No nosso artigo:** fonte das garantias da Tabela 1 e da fundamentação inteira.
- **O que ler:** as seções de NF e FF.

[2] **Dósa (2007)** — *The Tight Bound of First Fit Decreasing Is $\text{FFD}(I) \leq 11/9 \text{OPT}(I) + 6/9$* . ESCAPE 2007, LNCS 4614:1–11.

- **O que é:** fecha o problema aberto por 34 anos: a constante aditiva correta do FFD é $6/9$.
- **No nosso artigo:** a garantia 1.222 que confrontamos experimentalmente.
- **O que ler:** introdução e o enunciado do teorema.

[3] **Martello & Toth (1990)** — *Lower Bounds and Reduction Procedures for the Bin Packing Problem*. Discrete Applied Mathematics 28(1):59–70.

- **O que é:** define os limites inferiores **L1** e **L2** e o solver exato MTP.
- **No nosso artigo:** o L2 é o **denominador de todas as nossas razões empíricas** — sem essa referência o experimento não existe.

[4] **Kenyon (1996)** — *Best-Fit Bin-Packing with Random Order*. SODA '96:359–364.

- **O que é:** introduz o modelo **random-order**: razão **esperada** do BF entre 1.08 e 1.5, conjecturando ≈ 1.15 .
- **No nosso artigo:** ancoramos nossa motivação e o cronograma do 2º bimestre (experimento random-order pra dialogar com a conjectura).

PRIORIDADE 2 — recomendada

[5] **Falkenauer (1996)** — *A Hybrid Grouping Genetic Algorithm for Bin Packing*. J. of Heuristics 2(1):5–30. Fonte da distribuição `falkenauer_u120`.

[6] **Williamson & Shmoys (2011)** — *The Design of Approximation Algorithms*. Livro-texto.

[7] **Dósa, Li, Han & Tuza (2013)** — *Tight Absolute Bound for First Fit Decreasing*. Theoretical Computer Science 510:13–61.

PRIORIDADE 3 — contexto

[8] **Karmarkar & Karp (1982)** — *An Efficient Approximation Scheme for Bin-Packing*. FOCS '82. Aproximação quase-exata: $\text{OPT} + O(\log^2 \text{OPT})$.

[9] **Balogh et al. (2021)** — *A New Lower Bound for Classic Online Bin Packing*. SODA '21. Teto online: 1.5427.

[10] **Albers, Khan & Ladewig (2021)** — *Best Fit with Random Order Revisited*. Algorithmica. Razão $5/4$ no caso especial.

[11] **Hebbar, Khan & Sreenivas (2024)** — *Bin Packing under Random-Order: Breaking the Barrier of $3/2$* . arXiv:2401.04714. Limite inferior 1.144.

[12] **Ayyadevara et al. (2025)** — *Near-Optimal Algorithms for Stochastic Online Bin Packing*. ACM TALG. BF exatamente ótimo quando todos os itens $> 1/3$.

[13] **Garey & Johnson (1979)** — *Computers and Intractability*. Fonte da 3-PARTIÇÃO.

Novas referências da validação externa

[14] **Coffman, So, Hofri & Yao (1980)** — *A Stochastic Model of Bin-Packing*. Information and Control 44(2):105–115. Razão esperada exata $4/3$ para NF em $U(0,1]$.

[15] **Bentley et al. (1984)** — *Some Unexpected Expected Behavior Results for Bin Packing*. STOC '84. FF assintoticamente ótimo na expectativa; FFD excesso $O(1)$.

[16] **Bentley et al. (1983)** — *An Experimental Study of Bin Packing*. Allerton '83. Estudo experimental precursor que inspirou os teoremas de 1984.

10. Perguntas prováveis da defesa (com respostas curtas)

P: Por que L2 e não L1? R: $L1 = \lceil \text{soma} \rceil$ ignora fragmentação. L2 refina considerando itens grandes que exigem bin próprio e itens médios que não podem dividi-los com eles. Exemplo: $[0.6, 0.6, 0.6] \rightarrow L1=2, L2=3=OPT$. Usar L1 subestimaria o lower bound e **superestimaria** artificialmente as nossas razões de aproximação.

P: Sua razão $A/L2$ é a razão de aproximação real? R: Não — é conservadora. Como $L2 \leq OPT$, temos $A/L2 \geq A/OPT$. A qualidade real é **melhor** do que reportamos. Calcular A/OPT exigiria o ótimo exato, inviável em $n=16.000$ (NP-difícil). No 2º bimestre implementamos branch-and-bound pra instâncias pequenas e medimos contra OPT verdadeiro.

P: O que significa uma garantia ser "tight"? R: Que existe instância que a atinge — não é possível melhorar o fator. Johnson et al. provaram tightness para NF/FF/BF; Dósa para o $11/9+6/9$ do FFD.

P: De onde vem o $11/9$? R: Da análise do FFD: instâncias adversariais com itens em faixas específicas forçam o FFD a usar $11/9 \cdot OPT + 6/9$ bins, e Dósa provou que não piora mais que isso. O número é o resultado da análise — não tem intuição simples.

P: Por que a 3-partição é a distribuição mais difícil? R: Todos os itens em $(1/4, 1/2]$: no máximo 3 por bin e **nenhum item pequeno** para preencher as sobras (sem "argamassa"). As sobras ficam inutilizáveis e todas as heurísticas desperdiçam.

P: NF/FF/BF são online; FFD/BFD são offline. Qual a diferença? Por que importa? R: Online: decide na chegada do item, sem ver o futuro (modela streaming). Offline: vê tudo antes (pode ordenar). FFD/BFD precisam ser offline — ordenar exige a lista completa. A linha teórica online tem teto 1.5428 (Balogh et al. 2021): nenhuma heurística online bate isso.

P: Por que Rust? R: Performance comparável a C/C++ (ganha em 3 de 5, empata nos outros), segurança de memória sem coletor de lixo, e testes de propriedade first-class no tooling (**cargo test**) — foi um teste de propriedade que achou o contraexemplo FFD > FF.

P: Por que seu Rust ganhou do C? Não era pra C ser mais rápido? R: Com -O3 -march=native dos dois lados, a diferença está no **idioma**, não na linguagem: iteradores eliminam bounds check, **total_cmp** elimina branch de NaN, e o BF branchless vetoriza como **minpd** — o LLVM vetoriza melhor que o gcc nesse padrão. No FF/FFD o gcc empata. Zero **unsafe** no caminho.

P: E o epsilon 1e-9 no código? R: Tolerância de ponto flutuante: $0.1+0.2 \neq 0.3$ em IEEE 754. Sem tolerância, um item que "deveria" caber exatamente seria rejeitado por erro de arredondamento.

P: Por que mediana de 3 execuções e não média? R: Robustez contra outliers — qualquer pico de carga do sistema (outro processo) inflaria a média; a mediana ignora. E rodamos com a máquina ociosa: a 1ª rodada do benchmark foi descartada porque o Python mediu 25% mais lento com o sistema ocupado.

P: Se FFD tem garantia 1.222 e NF tem 2.0, por que alguém usaria NF? R: Tempo. NF é $O(n)$ e 170× mais rápido que FF em $n=16.000$, com qualidade ~30% pior. Em streaming com milhões de itens por segundo, NF é a escolha racional. O artigo mede exatamente esse trade-off.

P: O que você faria no 2º bimestre? R: (i) escalar até $n=64.000$ com tamanhos intermediários; (ii) solver exato branch-and-bound com L2 pra $n \leq 100 \rightarrow$ razão contra OPT verdadeiro; (iii) experimento random-order (BF em permutação aleatória) pra dialogar com a conjectura de Kenyon e o 1.144 de Hebbbar et al.; (iv) seções finais de Resultados/Discussão e Conclusão.

P: Onde a IA entrou no trabalho? R: 3 ferramentas — Hermes Agent (Longcat-2.0) implementou código, experimentos e texto; Claude Sonnet 4.6 revisou academicamente; Gemini 3.1 Pro revisou estilo e formalidade. Contribuição total estimada: ~101% da execução. Autoria intelectual e revisão final foram do autor.

P: Qual a contribuição original do artigo? R: O gap identificado na literatura: nenhuma referência lida apresenta (a) a redução formal da NP-dureza nem (b) avaliação **conjunta** de qualidade × tempo das implementações diretas nas distribuições de benchmark. Nosso experimento mede as duas dimensões lado a lado, reproduzível, código aberto — e documenta os contraexemplos FFD>FF.

11. Números para decorar (cola de defesa)

O quê	Valor
Garantia NF	$2 \cdot \text{OPT}$
Garantia FF / BF	$1.7 \cdot \text{OPT}$
Garantia FFD / BFD	$11/9 \cdot \text{OPT} + 6/9 \approx 1.222 \cdot \text{OPT}$

O quê	Valor
FFD médio medido (uniforme)	1.001
Pior razão empírica medida (3-partição)	1.24 (nenhuma passou disso)
NF vs FF tempo (n=16k)	NF 170× mais rápido
NF vs BFD tempo (n=16k)	NF 487× mais rápido
Execuções do experimento	1.000 (4 dist × 5 n × 10 reps × 5 algs)
Validação cruzada do bench	0 divergências em 9.000 comparações
Bench: rust/C no NF	0.57× (Rust 1.7× mais rápido)
Bench: Python vs compilados	~15× (linear), ~53–63× (quadráticos)
Problema aberto fechado por Dósa	34 anos (1973 → 2007)
Teto online (Balogh 2021)	1.5428
Conjectura Kenyon / limite Hebbbar	~1.15 / 1.144

12. O que já está pronto × o que falta

Pronto (1º bimestre):

- Código: 5 heurísticas + L2 + gerador + CLI + 11 testes (unit + integração + propriedade)
- Experimento: 1.000 execuções, **resultados.csv**, 3 gráficos
- Benchmark 4 linguagens: variantes bins-only, validação cruzada, mediana de 3 rodadas
- Artigo: todas as seções do 1º bimestre (.tex, .pdf) formatadas na SBC, incluindo **Considerações Parciais — 10 páginas**.
- Matriz de referências: **16** verificadas (14 peer-reviewed, 4 recentes, 2 semanais)
- Apresentação: 12 slides com tabela comparativa obrigatória
- Repo público com README completo + licença MIT sugerida + declaração de IA detalhada
- Declaração de IA: ~101% da execução; autor mantém autoria intelectual

Falta (2º bimestre, já no cronograma do artigo):

- Escala até n=64.000 + tamanhos intermediários (afinar regressão)
- Solver exato B&B com L2 ($n \leq 100$) → razão contra OPT verdadeiro
- Experimento random-order (conjectura de Kenyon, 1.144 de Hebbbar)
- Substituir as Considerações Parciais por uma **Conclusão** definitiva

Só você faz: ler as 4 referências de prioridade 1, subir o zip no Overleaf (ou usar o PDF compilado localmente), reler o texto final e ensaiar o timing (5–10 min).

13. Glossário rápido

Siglas do artigo

- **NF — Next Fit:** "Próximo que Cabe". Mantém só 1 bin aberto. Se o item não cabe, fecha o bin pra sempre e abre outro. É o algoritmo mais simples e mais rápido ($O(n)$), mas o que mais desperdiça espaço (garantia $2 \cdot \text{OPT}$).
- **FF — First Fit:** "Primeiro que Cabe". Para cada item, percorre todos os bins abertos do primeiro ao último e coloca no primeiro onde couber. Se nenhum servir, abre um novo. Mais lento ($O(n^2)$ na versão direta) porque varre tudo, mas desperdiça bem menos (garantia $1.7 \cdot \text{OPT}$).
- **BF — Best Fit:** "Melhor Encaixe". Para cada item, percorre todos os bins e escolhe aquele onde o item cabe e que tem o **menor espaço sobrando** (o bin mais cheio onde ainda cabe). A ideia é apertar o item no espaço mais justo possível, minimizando fragmentação. Mesma garantia e complexidade do FF ($1.7 \cdot \text{OPT}$, $O(n^2)$).
- **FFD — First Fit Decreasing:** "Primeiro que Cabe, Decrescente". Ordena todos os itens do **maior para o menor** e depois aplica o FF. A ideia: posicionar os grandes primeiro (eles definem a estrutura dos bins) e depois usar os pequenos como "argamassa" para tapar os buracos. Melhor garantia: $11/9 \cdot \text{OPT} + 6/9 \approx 1.222 \cdot \text{OPT}$.
- **BFD — Best Fit Decreasing:** "Melhor Encaixe, Decrescente". Ordena decrescente e aplica BF. Mesma garantia do FFD.
- **OPT(L):** o número mínimo absoluto de bins para a instância L. É o "gabarito" — se existisse um algoritmo perfeito, ele usaria OPT bins. Calcular OPT é NP-difícil.
- **L1:** lower bound trivial = [soma de todos os itens]. Se os itens somam 7.3, você precisa de pelo menos 8 bins (cada bin cabe no máximo 1.0 de soma). Fraco porque ignora que itens grandes não combinam entre si.
- **L2:** lower bound refinado de Martello e Toth. Considera que itens grandes demais para compartilhar bin com outros precisam de bin próprio. Sempre $\geq L1$ e $\leq \text{OPT}$. É o denominador das nossas razões empíricas.

Siglas e termos teóricos

- **NP:** classe de problemas cuja solução pode ser **verificada** rapidamente (em tempo polinomial), mas para os quais não se conhece algoritmo que **encontre** a solução rapidamente. Exemplo: verificar se uma partição em bins é válida é fácil (soma cada bin e checa ≤ 1); achar a partição ótima é que é difícil.
- **NP-difícil (NP-hard):** um problema é NP-difícil se todo problema em NP pode ser transformado nele. Informalmente: "pelo menos tão difícil quanto qualquer problema em NP". Bin Packing (otimização) é NP-difícil.
- **NP-completo (NP-complete):** é NP-difícil e está em NP. Bin Packing (versão de decisão: "dá com $\leq k$ bins?") é NP-completo.
- **"No sentido forte" (strongly NP-complete):** continua NP-completo mesmo quando os números da entrada são pequenos (escritos em unário). Consequência prática: não admite FPTAS (ver abaixo). A 3-PARTIÇÃO é NP-completa no sentido forte — por isso a redução dela para bin packing bloqueia FPTAS para bin packing também.
- **P vs NP:** pergunta em aberto mais famosa da computação. P = problemas que se resolvem rapidamente; NP = problemas que se verificam rapidamente. Se P = NP, tudo que se

verifica rápido também se resolve rápido (e não se precisaria de heurísticas). Crê-se que $P \neq NP$, mas ninguém provou.

- **Redução:** transformar um problema A em um problema B de modo que resolver B resolva A. Se A é sabidamente difícil e B é pelo menos tão difícil, B também é difícil. No artigo: reduzimos 3-PARTIÇÃO (sabidamente NP-completa) em Bin Packing → logo Bin Packing é pelo menos tão difícil quanto 3-PARTIÇÃO.
- **3-PARTIÇÃO:** dado um conjunto de $3m$ números que somam $m \cdot B$, com cada número entre $B/4$ e $B/2$, é possível dividi-los em m grupos de exatamente 3 números cada, todos somando exatamente B ? NP-completo no sentido forte. A restrição $B/4 < a_i < B/2$ garante que cada grupo tem exatamente 3 elementos (não 2 nem 4).
- **Razão de aproximação:** $A(L)/OPT(L)$. Se A usa 110 bins e OPT é 100, a razão é 1.10 (10% a mais que o ótimo). Quanto mais perto de 1, melhor o algoritmo.
- **p-aproximativo:** garantia de que $A(L) \leq p \cdot OPT(L) + c$ para toda instância L. O p é o fator multiplicativo (ex.: 1.7 para FF), e c é uma constante aditiva (ex.: 6/9 para FFD). Numa instância com $OPT = 1000$, FFD garante $\leq 1222 + 1 = 1223$ bins.
- **Tight (justo):** uma garantia é tight quando existe pelo menos uma instância que a atinge. Significa que não dá para melhorar o fator — ele é exato, não uma aproximação grosseira. Ex.: existe instância onde FF usa exatamente $1.7 \cdot OPT$ bins.
- **FPTAS (Fully Polynomial-Time Approximation Scheme):** um algoritmo de aproximação parametrizado por ϵ que, para qualquer $\epsilon > 0$, devolve solução a $(1+\epsilon)$ do ótimo em tempo polinomial em n e em $1/\epsilon$. É o "santo graal" da aproximação — quase-exato e rápido. Bin Packing no sentido forte **não admite** FPTAS (a menos que $P=NP$).
- **Lower bound (limite inferior):** um valor que sabemos ser $\leq OPT$. Serve como "chão" para avaliar algoritmos quando o OPT é incalculável. Se $A/LB = 1.02$, sabemos que A está a no máximo 2% do LB — e como $LB \leq OPT$, A está a no máximo 2% do OPT (pode estar ainda mais perto).
- **Online vs Offline:** online = o algoritmo vê um item por vez e decide sem saber o que vem depois (como uma fila); offline = o algoritmo vê todos os itens antes de decidir (pode ordenar, pode planejar). NF, FF, BF são online; FFD, BFD são offline.
- **Random-order:** modelo probabilístico onde os itens chegam online mas em ordem aleatória (permutação uniforme da lista). Fica entre o pior caso puro (adversário escolhe a ordem pior possível) e o caso médio i.i.d. (itens são sorteados independentemente).
- **i.i.d. (independentes e identicamente distribuídos):** cada item é sorteado da mesma distribuição de probabilidade, independentemente dos outros. Modelo usado na análise de caso médio (Coffman 1980, Bentley 1984).

Termos de implementação

- **Assintótico / notação $O(\cdot)$:** descreve como o tempo cresce com n . $O(n)$ = tempo proporcional a n (dobra n , dobra o tempo). $O(n^2)$ = tempo proporcional a n^2 (dobra n , quadruplica o tempo). $O(n \log n)$ = entre linear e quadrático (é o custo de ordenar — merge sort, por exemplo).
- **Constante multiplicativa:** o fator escondido no $O(\cdot)$. Dois algoritmos $O(n^2)$ podem ter tempos muito diferentes: se um faz $n^2/2$ operações e outro faz $5n^2$, o segundo é $10\times$ mais

lento mas ambos são $O(n^2)$. O benchmark de linguagens mede essa constante.

- **LTO (Link-Time Optimization):** otimização que o compilador Rust faz **na hora de juntar** todos os pedaços do programa (crates). Sem LTO, cada pedaço é otimizado separadamente; com LTO, o compilador vê tudo junto e otimiza melhor (ex.: pode eliminar chamadas de função que cruzam crates).
- **Vetorização (SIMD — Single Instruction, Multiple Data):** a CPU moderna consegue processar 2, 4, ou 8 números de uma vez só numa única instrução. Ex.: `minpd` compara 2 doubles em paralelo. Se o código é escrito sem `if` desnecessários (branchless), o compilador consegue usar essas instruções automáticas. No nosso BF, a busca pelo bin com menor sobra é vetorizada porque é escrita como uma redução de mínimo sem branch.
- **Bounds check:** o Rust, por segurança, insere verificação de limites em todo acesso por índice (`array[i]` — "i está dentro do array?"). Isso custa ciclos. Quando se usa iteradores em vez de indexação, o compilador sabe que os limites são respeitados e **elimina** a verificação.
- **Branchless:** código sem desvios condicionais (`if/else`). O CPU moderno funciona com "pipeline" (prepara instruções adiantado); um `if` pode fazer o pipeline errar a previsão e jogar trabalho fora. Código branchless evita isso. No nosso BF, a comparação "cabe?" e "é o menor resíduo?" são feitas com aritmética em vez de `if` .
- **total_cmp** : método do Rust para comparar floats (f64) pelos bits, sem tratar NaN como caso especial. A comparação normal de float precisa de um branch pra NaN; **total_cmp** converte pra inteiro e compara — branchless, mais rápido no sort.
- **splitmix64:** um algoritmo determinístico para "misturar" bits de uma semente. Dado um número, produz outro que parece aleatório mas é 100% reproduzível. Usado no gerador para misturar a semente com o tamanho n, garantindo que cada combinação (semente, n) gera uma instância única mas reproduzível.
- **IEEE 754:** o padrão de como computadores representam números com ponto flutuante (float/double). Uma consequência famosa: $0.1 + 0.2 = 0.30000000000000004$, não 0.3. Por isso o código usa tolerância (epsilon = $1e-9$) ao verificar se um item "cabe" num bin — sem isso, um item que deveria caber exatamente seria rejeitado.

14. Os algoritmos passo a passo (para entender de verdade)

Esta seção explica cada algoritmo com **dois exemplos** — um simples onde todos acertam, e um que revela as diferenças.

Exemplo 1: $L = [0.7, 0.6, 0.4, 0.2]$ — $OPT = 2$

O ótimo é $\{0.7, 0.2\} + \{0.6, 0.4\} = 2$ bins (soma de cada um ≤ 1).

Exemplo 2: $L = [0.3, 0.8, 0.5, 0.4, 0.7, 0.2]$ — $OPT = 3$

O ótimo é $\{0.8, 0.2\} + \{0.7, 0.3\} + \{0.5, 0.4\} = 3$ bins.

14.1 Next Fit (NF) — "Só olha o bin atual"

Regra: mantém UM ÚNICO bin aberto. Se o item cabe, coloca lá. Se não cabe, **fecha** esse bin pra sempre (nunca mais volta nele) e abre um novo.

Exemplo 1:

```
Item 0.7 → bin1 [0.7] (sobra 0.3)
Item 0.6 → bin1 sobra 0.3, não cabe → FECHA bin1, abre bin2
[0.6] (sobra 0.4)
Item 0.4 → bin2 sobra 0.4, cabe → bin2 [0.6, 0.4] (sobra 0.0)
Item 0.2 → bin2 sobra 0.0, não cabe → FECHA bin2, abre bin3
[0.2]
Total: 3 bins (OPT = 2) x
```

Exemplo 2:

```
Item 0.3 → bin1 [0.3] (sobra 0.7)
Item 0.8 → bin1 sobra 0.7, não cabe → FECHA bin1, abre bin2
[0.8] (sobra 0.2)
Item 0.5 → bin2 sobra 0.2, não cabe → FECHA bin2, abre bin3
[0.5] (sobra 0.5)
Item 0.4 → bin3 sobra 0.5, cabe → bin3 [0.5, 0.4] (sobra 0.1)
Item 0.7 → bin3 sobra 0.1, não cabe → FECHA bin3, abre bin4
[0.7] (sobra 0.3)
Item 0.2 → bin4 sobra 0.3, cabe → bin4 [0.7, 0.2] (sobra 0.1)
Total: 4 bins (OPT = 3) x — desperdiçou bin1 que ficou com só
0.3
```

Por que é rápido: para cada item, faz UMA verificação (cabe no bin atual?). Total: n verificações → $O(n)$.

Por que desperdiça: nunca volta num bin que fechou. O bin1 ficou com 0.3 de espaço vazio — se pudesse voltar, o 0.2 caberia lá.

Garantia: $NF(L) \leq 2 \cdot OPT(L)$. Nunca usa mais que o dobro do ótimo. **Intuição da prova:** dois bins consecutivos (o que fechou + o que abriu) sempre somam mais que 1 (senão o item teria cabido e o bin não teria sido fechado). Então cada "par" ocupa mais que 1 → no máximo $2 \times$ o necessário.

14.2 First Fit (FF) — "Percorre todos do primeiro ao último"

Regra: para cada item, percorre todos os bins abertos **do primeiro ao último** e coloca no **primeiro** onde couber. Se nenhum servir, abre um novo.

Exemplo 1:

```

Item 0.7 → nenhum bin aberto → abre bin1 [0.7] (sobra 0.3)
Item 0.6 → bin1 sobra 0.3, não cabe → abre bin2 [0.6] (sobra 0.4)
Item 0.4 → bin1 sobra 0.3, não cabe; bin2 sobra 0.4, cabe! → bin2 [0.6, 0.4]
Item 0.2 → bin1 sobra 0.3, cabe! → bin1 [0.7, 0.2]
Total: 2 bins = OPT ✓

```

Exemplo 2:

```

Item 0.3 → abre bin1 [0.3] (sobra 0.7)
Item 0.8 → bin1 sobra 0.7, não cabe → abre bin2 [0.8] (sobra 0.2)
Item 0.5 → bin1 sobra 0.7, cabe! → bin1 [0.3, 0.5] (sobra 0.2)
Item 0.4 → bin1 sobra 0.2, não; bin2 sobra 0.2, não → abre bin3 [0.4] (sobra 0.6)
Item 0.7 → bin1 não; bin2 não; bin3 sobra 0.6, não → abre bin4 [0.7] (sobra 0.3)
Item 0.2 → bin1 sobra 0.2, cabe! → bin1 [0.3, 0.5, 0.2]
Total: 4 bins (OPT = 3) ✗ – melhor que NF? Não neste caso (ambos deram 4)

```

Por que é mais lento: para cada item, pode ter que verificar TODOS os bins abertos. Se há k bins abertos e n itens, no pior caso faz $n \times k$ verificações. Como k pode chegar a $\sim n$, é $O(n^2)$.

Garantia: $FF(L) \leq 1.7 \cdot OPT(L)$. Nunca usa mais que 70% a mais que o ótimo.

14.3 Best Fit (BF) — "Aperta no mais cheio"

Regra: percorre TODOS os bins abertos e escolhe aquele onde o item cabe e que tem o **menor espaço restante** (o bin mais cheio onde ainda cabe). Empate: pega o primeiro.

Diferença do FF: o FF pega o **primeiro que serve**; o BF pega o **mais justo que serve**.

Exemplo 2 (mostrando a diferença):

```

Item 0.3 → abre bin1 [0.3] (sobra 0.7)
Item 0.8 → bin1 sobra 0.7, não cabe → abre bin2 [0.8] (sobra 0.2)

```

```

Item 0.5 → bin1 sobra 0.7, cabe; bin2 sobra 0.2, não cabe
           → Só bin1 serve → bin1 [0.3, 0.5] (sobra 0.2)
Item 0.4 → bin1 sobra 0.2, não; bin2 sobra 0.2, não → abre bin3
           [0.4] (sobra 0.6)
Item 0.7 → bin1 não; bin2 não; bin3 sobra 0.6, não → abre bin4
           [0.7] (sobra 0.3)
Item 0.2 → bin1 sobra 0.2, cabe (restaria 0.0);
           bin2 sobra 0.2, cabe (restaria 0.0);
           bin4 sobra 0.3, cabe (restaria 0.1);
           → Menor sobra: bin1 ou bin2 (ambos 0.0) → pega bin1
           → bin1 [0.3, 0.5, 0.2]
Total: 4 bins — mesmo resultado que FF neste caso

```

Nota: FF e BF frequentemente dão o mesmo resultado. A diferença aparece em instâncias maiores. Na prática, BF tende a ser levemente melhor em qualidade mas mais lento (porque precisa calcular a sobra de todos os bins, não só achar o primeiro que serve).

Garantia: $BF(L) \leq 1.7 \cdot OPT(L)$ — mesma do FF.

14.4 FFD e BFD — "Ordena antes, depois aplica FF ou BF"

Regra FFD: ordena os itens do maior para o menor, depois aplica FF. **Regra BFD:** ordena do maior para o menor, depois aplica BF.

Por que ordenar ajuda (na maioria dos casos): itens grandes são os mais difíceis de encaixar (ocupam muito espaço). Se eles chegam primeiro, definem a "espinha dorsal" dos bins. Depois, itens pequenos funcionam como "argamassa" preenchendo os buracos.

Exemplo 2 com FFD:

```

Ordenado: [0.8, 0.7, 0.5, 0.4, 0.3, 0.2]
Item 0.8 → bin1 [0.8] (sobra 0.2)
Item 0.7 → bin1 não → bin2 [0.7] (sobra 0.3)
Item 0.5 → bin1 não; bin2 não → bin3 [0.5] (sobra 0.5)
Item 0.4 → bin1 não; bin2 sobra 0.3, não; bin3 sobra 0.5, cabe!
           → bin3 [0.5, 0.4] (0.1)
Item 0.3 → bin1 não; bin2 sobra 0.3, cabe! → bin2 [0.7, 0.3]
Item 0.2 → bin1 sobra 0.2, cabe! → bin1 [0.8, 0.2]
Total: 3 bins = OPT ✓ — a ordenação ajudou!

```

Garantia: $FFD(L) \leq 11/9 \cdot OPT(L) + 6/9$. $11/9 \approx 1.222$. Melhor garantia entre as 5.

Mas atenção — ordenar pode PIORAR! Existem instâncias reais (encontradas nos nossos testes) onde FFD usa MAIS bins que FF sem ordenar. A intuição: quando todos os itens são de tamanho

médio (entre 0.26 e 0.50), ordenar espalha os maiores no começo criando sobras pequenas demais para os que vêm depois. Sem ordenar, a mistura casual pode combinar melhor por acaso.

14.5 Resumo visual das diferenças

Algoritmo	O que faz	Vantagem
Desvantagem		
NF	Só olha o bin atual	Muito rápido $O(n)$
Desperdiça muito ($2 \cdot OPT$)		
FF	Percorre todos, pega 1º	Bom equilíbrio
Lento $O(n^2)$		
BF	Percorre todos, pega melhor	Menos fragmentação
Lento $O(n^2)$		
FFD	Ordena + FF	Melhor garantia
Lento + precisa ver tudo		
bfd	Ordena + BF	Melhor garantia
mais lento de todos		0

15. Fala de abertura (30s, decorada)

"Meu artigo estuda o bin packing unidimensional, que é NP-difícil. Implementei 5 heurísticas clássicas em Rust, cada uma com sua garantia teórica de pior caso — de $2 \cdot OPT$ no Next Fit até $11/9 \cdot OPT + 6/9$ no First Fit Decreasing, fechado por Dósa em 2007 após 34 anos aberto. A pergunta: quão pessimistas são essas garantias na prática? Rodei 1.000 execuções controladas e a resposta é: muito — o FFD médio ficou em 1.001 contra a garantia de 1.222. Descobri contraexemplos onde ordenar piora o resultado, e fiz um benchmark secundário em 4 linguagens mostrando que a mesma assintótica esconde constantes de até $60\times$."

16. Declaração de IA: como apresentar na defesa

Se perguntar, lê direto da tabela do artigo (§7):

Ferramenta	O que fez	O que eu fiz
Hermes Agent (Longcat-2.0)	Estruturou projeto, implementou código (Rust + bench C/C++/Python), rodou 1.000 experimentos, gerou gráficos, redigiu 1ª versão do texto	Escolheu/validou tema; descartou rodada contaminada; revisou texto; verificou testes; leu referências

Ferramenta	O que fez	O que eu fiz
Claude Sonnet 4.6 (Thinking)	Revisão acadêmica: avaliou clareza, coesão, metodologia, formato SBC, 16 refs; gerou relatório; aplicou correções aprovadas	Selecionei e aprovei cada modificação individualmente antes da edição
Gemini 3.1 Pro (High)	Revisão de estilo (elevou coesão/tom); adição das Considerações Parciais; formatação de LaTeX avançado (<code>tabularx</code>) para a página 10; compilação do <code>.pdf</code> via terminal	Analisei feedback crítico, exigi a quebra do texto em 10 páginas para encaixar nas regras da SBC, e validei a nova tabela acadêmica

Declaração oral (30s): *"Três ferramentas de IA contribuíram: Hermes Agent implementou código e texto, Claude Sonnet revisou academicamente, e Gemini revisou estilo e formalidade. A contribuição total é estimada em ~101% — a execução foi quase toda da IA, mas a autoria intelectual, a direção e a revisão final são minhas. Nenhum arquivo foi alterado sem minha aprovação prévia."*

Guia gerado em 24/09/2026 como material de estudo para a defesa. Atualizado com as correções do Gemini 3.1 Pro (estilo), Claude Sonnet 4.6 (acadêmica) e Claude Opus 4.6 Thinking (expansão do glossário, seção 14 de algoritmos passo a passo, e correções ortográficas). Fontes: o próprio artigo (artigo-parcial.tex), o código do repositório e as verificações bibliográficas (Crossref/arXiv/DBLP) feitas durante o projeto.